GlucoTwin

Architecture brief | Digital Twin Challenge 2026

Purpose and boundary

A local proof of concept forecasts synthetic glucose values two hours ahead using fictional patient context and a simulated 15-minute sensor stream. The software is for engineering demonstration and education only; no real patient records, clinical validation, or treatment recommendations are included.

Data flow

Stage	What happens	Artifact
Generate	Seeded fictional demographics, BMI, HbA1c and time series; glucose, heart rate, activity, sleep and cyclic time features.	patients.csv sensor_readings.csv
Align	Sort each patient stream, require exact 15-minute intervals, and shift glucose eight samples to form t + 120 min target.	training frame
Evaluate	Chronological pooled timestamp split: first 80% for fit, latest 20% for test. Compare with current-glucose persistence.	metrics.json
Serve	Streamlit selects a synthetic patient, advances the virtual stream, builds features and displays a local model estimate.	app.py
Review	Tests cover deterministic generation, horizon alignment, chronological training and invalid cadence.	tests/

Model and measured synthetic run

Target: glucose (mg/dL) at t + 120 minutes. This checked-in run used NumpyRidge (offline fallback; install requirements for scikit-learn model), with 96,420 training rows and 24,060 test rows. Model MAE: 5.048 mg/dL. Last-value baseline MAE: 8.567 mg/dL. Split cutoff: 2026-10-15T17:30:00+00:00. These figures reflect only the synthetic generator.

Reproducibility and limitations

Run python generate_data.py, python train_model.py, then streamlit run app.py. A seeded NumPy ridge fallback writes the artifact in offline environments; with declared dependencies installed, training uses scikit-learn HistGradientBoostingRegressor and joblib. The exact model choice is recorded in metrics. Synthetic distributions do not establish real-world accuracy, safety, equity, or usefulness. Do not use for patient care.

System requirements

Python with pandas, NumPy, scikit-learn, Streamlit, Plotly, pytest and joblib. The offline fallback additionally uses the Python standard library. No cloud services or external health-data connections.